

SecMdp: Towards Privacy-Preserving Multimodal Deep Learning in End-Edge-Cloud

Zhao Bai*, Mingyue Wang[†], Fangda Guo[‡], Yu Guo*, Chengjun Cai[§], Rongfang Bie*, Xiaohua Jia[¶]

*Beijing Normal University, [†]Harbin Institute of Technology, Shenzhen

[‡]CAS Key Laboratory of AI Safety & Security, Institute of Computing Technology, Chinese Academy of Sciences

[§]City University of Hong Kong, Dongguan Research Institute, [¶]City University of Hong Kong

zhaobai@mail.bnu.edu.cn, mingyue.wang@my.cityu.edu.hk,

guofangda@ict.ac.cn, yuguo@bnu.edu.cn, chengjun.cai@cityu-dg.edu.cn,

rfbie@bnu.edu.cn, csjia@cityu.edu.hk

Abstract—Multimodal deep learning technologies have advanced significantly, which brings extensive applications in diverse fields. The substantial computational demands of training and prediction in multimodal deep learning have made the End-Edge-Cloud (EEC) framework popular. It is essential to protect multimodal data and model privacy in such a framework. However, traditional cryptographic methods, though secure for data and models at edge nodes, cause efficiency limitations. In this paper, we propose SecMdp, an SGX-assisted secure computational framework for multimodal data in the EEC architecture. Edge nodes are equipped with the trusted execution environment (e.g., Intel SGX) to run multimodal algorithms. Additionally, to address the side-channel attacks of SGX, we present an enhanced PathORAM algorithm, MM_PathORAM, for the multimodal training and prediction processes, which are tailored for multimodal deep learning scenarios. It accelerates multimodal data access while protecting data privacy and model security. Experimental evaluation supports the effectiveness of our design in preserving edge computing efficiency. It demonstrates negligible impact on the speed of multimodal data loading, the configuration of model parameters during training, or the accuracy of predictions.

Index Terms—Multimodal deep learning, End-Edge-Cloud, SGX, PathORAM

I. INTRODUCTION

With the prosperity of multimodal deep learning and Internet of Things (IoT) technologies, their applications have become increasingly diverse and popular nowadays, such as ChatGPT [1], visual dialogue [2], autonomous driving [3], and image-text search [4]. In these scenarios, multimodal data of users is collected with IoT devices. Deep learning algorithms are then applied to train models on a cloud server, which will provide abundant prediction services to users.

Despite the promising advancement, multimodal deep learning suffers from efficiency issues. As mobile IoT devices surge, the data for training and prediction explodes, which escalates computational demands for multimodal model services. As such, it becomes impractical for a single cloud server to handle such large-scale datasets. To speed up the multimodal deep learning process, the concept of edge intelligence [5]–[7] is introduced into this field. As shown in Fig. 1, the End-

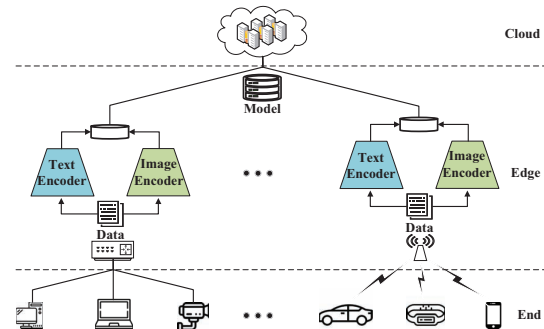


Fig. 1. Multimodal model services based on the EEC architecture.

Edge-Cloud architecture is used to deploy multimodal model services. Specifically, mobile IoT data is uploaded to the edge nodes for training, which is then transmitted to the cloud server for aggregation. Then, each edge node conducts the prediction computation for its users. This design harnesses the computational and transmission advantages of edge computing, which accelerates both the training and prediction processes for multimodal deep learning.

With the increasing concern about data privacy, it is significant to provide security guarantees for practical multimodal systems. While the above EEC architecture seems ideal for multimodal models, several critical privacy issues remain to be solved. First, there is no privacy protection for data from the end nodes during the multimodal services since data is transmitted in the plaintext form. However, when data is protected by traditional cryptographic encryption methods, it is hard to train and predict data efficiently and accurately. Second, the model and data on the edge nodes are vulnerable to many attacks, such as Man-In-The-Middle (MITM) attacks [8], data poisoning attacks [9], training data inference attacks [10], and model inversion attacks [11], which can damage the system. Third, it lacks a secure multimodal deep learning design for data privacy and model security protection without degrading efficiency and accuracy. Existing related solutions [7], [12] proposed to use the Trusted Execution Environment (TEE) (e.g., Intel SGX) technology to achieve

Co-primary authors: Zhao Bai, Mingyue Wang. Corresponding author: Rongfang Bie, Yu Guo and Fangda Guo.

this goal. However, in these designs, the edge nodes are susceptible to side-channel attacks [13] during data interactions, which may compromise the multimodal deep learning training process and risk user privacy [14]. Although the traditional PathORAM [15] algorithm can defend against side-channel attacks, it is time-consuming when loading multimodal data. Thus, the PathORAM algorithm cannot be applied directly in multimodal systems.

In this paper, we propose SecMdp, a secure multimodal deep learning framework in the EEC architecture with SGX enclave. It utilizes the EEC architecture to boost the efficiency of training and prediction processes in multimodal deep learning models. To protect data privacy and model security, SGX enclave is deployed on each edge node. Further, we redesign the Multimodal Path Oblivious Random Access Machine (MM_PathORAM) algorithm, which can mitigate the risks of side-channel attacks on SGX for multimodal applications. Particularly, the MM_PathORAM algorithm is devised into a read-only variant since the IoT data is accessed only once during multimodal training and prediction processes. Thus, it can achieve fast data access while protecting the security of user information. Main contributions can be summarized as:

- We propose SecMdp, a multimodal deep learning framework based on the EEC architecture, where we establish an SGX-assisted trusted multimodal computing environment at the edge node.
- We design the MM_PathORAM algorithm for multimodal data applications. It can prevent information leakage from side-channel attacks on SGX while improving the efficiency of data loading during multimodal prediction and training processes.
- We implement the system prototype and deploy it on the edge nodes. Experimental results show that our framework does not influence the speed of multimodal data loading, the loss during model training, or the accuracy of predictions while guaranteeing model security.

The remainder of this paper is structured as follows: Section II reviews related work. Section III introduces the system architecture and the threat model of the system. We describe the design of SecMdp in Section IV. Section V describes the construction of SecMdp. Section VI conducts a security analysis of the SecMdp. Section VII evaluates the SecMdp framework. Finally, we conclude our paper in Section VIII.

II. RELATED WORK

Our design is framed based on the EEC architecture, focusing on three technologies: Multimodal Deep Learning, Trusted Execution Environment (TEE), and Path Oblivious Random Access Machine (PathORAM). In this section, we review some state-of-the-art works and developments in these fields.

A. Multimodal Deep Learning

Multimodal deep learning, a specialized branch within artificial intelligence, processes information of multiple modalities, including text, images, videos, and audio. It finds extensive applications in fields such as visual question answering [16],

visual dialogue [17], visual commonsense reasoning [18], and image-text retrieval [19]. Nevertheless, the multimodal training and prediction processes are accompanied by potential privacy risks. To address these issues, researchers have utilized Homomorphic Encryption (HE) [20], [21] and Differential Privacy (DP) [22], [23] techniques to protect data privacy during the deep learning training and prediction processes (as compared in Table I). However, the secure computing frameworks relying on homomorphic encryption are inefficient, and those based on differential privacy technology may reduce data accuracy due to the noise. Thus, we introduce a multimodal deep learning framework with the assistance of SGX, aiming to enhance the security and efficiency of training and prediction processes [24] simultaneously.

B. TEE

The TEE technology offers a secure setting for conducting multimodal training and prediction processes. It guarantees data and program integrity and confidentiality by isolating CPU hardware. This prevents unauthorized access or alterations by operating systems, super users, or hardware-based attacks. A notable implementation of this technology is Intel SGX, a trusted hardware solution. SGX features an enclave, a physically isolated area for key storage and code execution, ensuring data protection even if the platform is compromised [25]. To enhance the efficiency of multimodal deep learning within the SGX, researchers [26] utilize the Occlum Library Operating System (LibOS). It is a multi-process LibOS that integrates Intel SGX, which is acclaimed for its security, efficiency, and extensive compatibility. Researchers have developed deep learning frameworks utilizing Intel SGX, exemplified by Lasagna [7] and Occlumency [12]. These frameworks execute multimodal training and prediction computations within the SGX enclave, which guarantees the integrity and confidentiality of multimodal data and programs. It is also able to defend prevalent attack methods in multimodal deep learning, including data poisoning attacks [9], training data inference attacks [10], model stealing attacks [27], and model inversion attacks [11]. Nonetheless, it remains susceptible to side-channel attacks. The necessity of accessing data from untrusted regions during multimodal training and prediction processes exposes the data to such attacks during its transfer to the enclave. These attacks can manifest as syscall snooping, page faults, or cache-based attacks [14], among others. To mitigate these vulnerabilities, we redesign the PathORAM algorithm [15] for SGX to defend against side-channel attacks.

C. PathORAM

PathORAM [15] algorithm represents a simple yet highly efficient ORAM [28] technique, serving as an encryption scheme that can hide the data access patterns in IO operations. Its principle is to randomly change the data storage location with each access, which makes each interaction unpredictable and unrelated. The algorithm can successfully hide the actual access patterns of the user. It re-encrypts data using a random

TABLE I
COMPARISON OF PRIVACY-PRESERVING DEEP LEARNING SCHEMES
Rotations: ROTATION OPERATIONS IN HOMOMORPHIC CRYPTOGRAPHIC CONVOLUTION; *Mult Depth*: THE NUMBER OF LAYERS OF THE MULTIPLICATION OPERATION PERFORMED; $Lap(x|b)$: THE PROBABILITY DENSITY FUNCTION OF THE LAPLACE DISTRIBUTION; h^* : FINDING THE OPTIMAL PRIVACY BUDGET HYPERPARAMETER

Scheme	Method	Multimodal	Datasets	Accuracy	Data Privacy	Efficiency
[20]	HE	✗	CIFAR10 (100) ImageNet	-0.05% (-0.18)% -0.17%	Fully Homomorphic Encryption	$Rotations : B - 1 + 4 \log_2 w$ $Mult Depth : 3$
Falcon [21]	HE	✗	CIFAR10 MNIST	0% 0%	Homomorphic Encryption Block-Circulant Matrices	$Rotations : B - 1$ $Mult Depth : 1$
DPTRNN [22]	DP	✗	KTH UCF50	-5% -5%	Laplace	$Lap(x b) = \frac{1}{2b} \exp\left(-\frac{ x }{b}\right)$
PriMonitor [23]	DP	✓	CH-SIMS CMU-MOSI	-8% -7.4%	Gaussian	$h^* = \operatorname{argmax}_{h \in H} \text{Fusion}(h)$

encryption algorithm, further securing data safety. However, despite the fact that it can guarantee data privacy, its data retrieval is ineffective. Hence, it fails to meet the needs of practical application scenarios.

To optimize the efficiency of the PathORAM algorithm, there have been numerous related research developments. Chen *et al.* [29] introduced the Titanium system, a metadata-hidden file-sharing solution that enhances file read speeds while preserving data confidentiality and integrity. Rajat *et al.* developed LAORAM [30], which is designed to protect user privacy during embedding tables training. It enhances the efficiency of the original PathORAM algorithm by five times. Raouf *et al.* contributed IR-ORAM [31], which targets a reduction of memory footprint to boost its efficiency based on PathORAM algorithm. Wang *et al.* proposed D-ORAM [32], providing high-level privacy protection and minimal operational interference on cloud servers with untrusted memory. Despite these significant advancements on read efficiency in the PathORAM algorithm, its file reading speed remains inadequate for the data reading demands in the multimodal deep learning scenarios. Thus, we propose a new PathORAM design, MM_PathORAM, given that the dataset is read only once during both training and prediction processes. It is designed for multimodal deep learning, where it requires that the data read speed matches the model training speed.

III. PROBLEM FORMULATION

A. System Architecture

The structure of our multimodal deep learning system is depicted in Fig. 2, including three components: *cloud server*, *edge node*, and *end node*. Detailed descriptions of each component are as follows:

The **cloud** server consists of a network of multiple servers. Its principal role involves deploying the system architecture, receiving encrypted multimodal models from the edge node, and performing model aggregations.

The **edge** node needs to receive, train and predict multimodal data during model processing. To preserve the privacy of user data and the security of multimodal model, SGX enclave is strategically implemented at the edge node in our

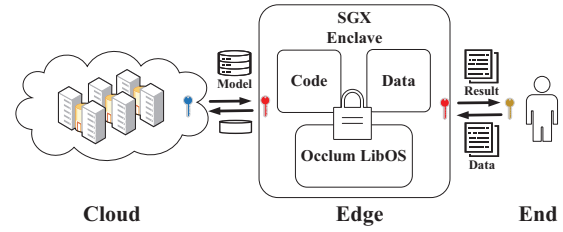


Fig. 2. System architecture.

design, where the model will be processed in SGX. There are two processes at the edge node: training and prediction. In the multimodal training process, the edge node receives user data from the end node and processes it using a multimodal algorithm. Then, it forwards the trained model to the cloud server for aggregation. Upon completion of aggregation, the cloud server transmits the aggregated model back to the edge node to facilitate prediction. In the multimodal prediction process, the model loads the prediction data from the end node and conducts prediction operations. Finally, the prediction results are returned to the end node.

The **end** node consists of mobile devices and IoT gadgets. The fundamental role of end node is to gather multimodal data from users and forward it to the edge node for further multimodal training and prediction. All data is encrypted before transmitted to the edge node.

The problem of our concern is that, given the EEC-based multimodal deep learning architecture, the edge nodes can provide efficient model services while preserving data privacy and system security. Since the edge node is equipped with the SGX enclave, the system security shall consider these issues about the SGX.

B. Threat Model

In the EEC-based multimodal deep learning systems, we make the following threat assumptions.

The cloud server is characterized as semi-honest. It honestly aggregates models from edge nodes while it attempts to access the multimodal data collected by end nodes.

The edge node that performs the main multimodal deep learning operations is considered semi-honest. The data training and prediction are processed in SGX, while all data from end nodes is in the ciphertext form. The edge node is eager to gain data information during the model processing.

The end node that provides training and prediction data is assumed to be semi-honest. It provides the collected multimodal data for model generation and enjoys the prediction services following the predefined protocols. However, it intends to obtain the model information from the cloud server and edge nodes.

There are also outsider attackers for the EEC-based deep learning system. During the data and model transmission, the EEC architecture faces the threat of MITM attacks, where attackers can intercept the data in transmission. Throughout the multimodal training and prediction processes, edge nodes encounter various security challenges, such as training data inference attacks, data poisoning attacks, model stealing attacks, and model inversion attacks. All of them pose risks to data privacy or model security, which can be effectively defended with SGX. However, SGX is vulnerable to side-channel attacks. The multimodal training is performed within the SGX enclave, with training data loaded from local disks. This transition from non-trusted to trusted regions brings vulnerabilities to side-channel attacks. The most representative ones are cache-based and page fault attacks. The cache-based attacks [33] exploit the caching behavior within CPUs to infer operations on data or instructions. The page fault attacks [34] rely on monitoring page faults to infer if a victim has accessed specific memory locations, a strategy that can be used to extract encryption keys. In this paper, we exclude more complex side-channel attacks like power monitoring attacks and electromagnetic attacks.

Remarks. In the proposed EEC-based systems, each edge node works as an intermediary between the cloud server and end nodes, highlighting the importance of edge computing security. Thus, we focus on the security of edge rather than the security of the model aggregation process at the cloud server, which has been discussed in some previous works [35]–[37].

IV. DESIGN OF SECMDP

In this section, we propose the SecMdp framework to defend against security risks in the EEC architecture. We first introduce its principles and overall framework. We also present the MM_PathORAM algorithm and analyze its theoretical basis and time complexity. We introduce the notations used in this paper as shown in TABLE II.

A. Overview

In the EEC architecture, the SecMdp framework utilizes SGX in the edge nodes to protect data privacy during multimodal data loading, training, model aggregation, and prediction phases. It further applies the MM_PathORAM algorithm to counter side-channel attacks when SGX loads data from untrusted areas. The MM_PathORAM algorithm relies on SGX to secure the position map, and the integration of SGX

TABLE II
NOTATIONS

Symbol	Definition
n	size of a dataset
k	depth of the binary tree
d_i	a image-text pair data
$d[k]$	the collection of k datasets from the leaf node to the root node
$d[n]$	all datasets stored on the disk
B_i	bucket in MM_PathORAM
R_i	real block in B_i
U_i	dummy block in B_i
$B_i \leftarrow R_i + U_i$	the storage of the real data block R_i mixed with dummy blocks in a single B_i
$P[d_i]$	the path from a leaf node where data is located to the root node
$PM[n]$	position map for storing data
$PM[n] \leftarrow B$	write the data block into the position map
Oram.access ($PM[n], d_i$)	retrieve the path $P[d_i]$ where d_i is located in position map

and the MM_PathORAM algorithm enhances the security of the EEC architecture.

The proposed SecMdp framework is depicted in Fig. 3, and the workflow has four key segments: data loading, data training, model aggregation, and data prediction. In the SecMdp framework, data is decrypted only inside the enclave and stored encrypted outside the enclave. Following system initialization, the data loading phase begins with the edge node requesting data from the end node. The end node responds by transmitting encrypted data to the edge node through a secure channel established using the Occlum LibOS, guaranteeing the data transmission security. Before data training, the SGX reads data from the disk using the MM_PathORAM algorithm, decrypts it within the enclave, and then uses the plaintext for training. The details of this process will be discussed in Section V-A. Then, it is uploaded to the cloud server in an encrypted form for model aggregation. The cloud server will return the aggregated model to the edge node for data prediction. In the data prediction phase, the end node encrypts and uploads the data, waiting to proceed to the edge node. The enclave then loads the data from the disk using the MM_PathORAM algorithm. To protect security, the prediction algorithm and model data are securely processed within Occlum. The edge node returns the prediction results to the end node. We will discuss more about this process in Section V-B.

B. MM_PathORAM

The MM_PathORAM is an algorithm specifically designed for multimodal deep learning. It effectively defends against side-channel attacks when SGX-assisted multimodal deep learning algorithms load data. There are several steps for the multimodal algorithm to read the local dataset. It first reads the storage paths of the datasets. Then, it retrieves all image

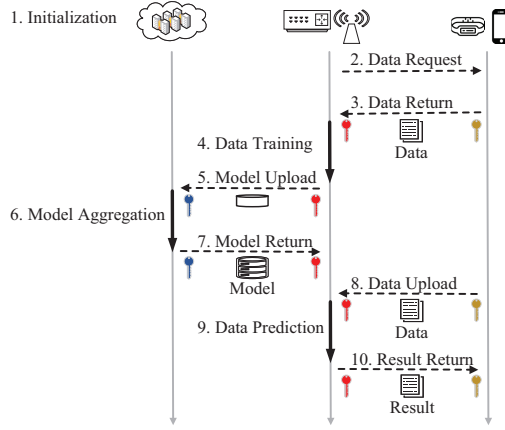


Fig. 3. Overview of SecMdp framework protocol. Solid lines indicate computational processes, while dashed lines represent wireless communication.

and text data stored in these paths and pairs the image and text data. Finally, the algorithm converts image-text data into tensor information for the multimodal training and prediction network. However, multimodal algorithms are vulnerable to side-channel attacks during the data loading phase, so the researchers employ PathORAM [15] to resist such attacks. While PathORAM offers a high level of security, its data reading speed is considerably slow. It is essential to maintain the dataset reading speed to match with the model training and prediction speed for training and prediction processes. To address this issue, we propose the MM_PathORAM algorithm, which is designed to enhance efficiency in multimodal tasks.

In our MM_PathORAM design, data is stored in the binary tree structure. Upon data access, the system reads the path from the root to the leaf where the data resides. This mechanism ensures that external entities cannot infer user information through cache-based attacks or page fault attacks. Building upon PathORAM, MM_PathORAM eliminates writing operations to enhance reading efficiency, as the multimodal training and prediction processes solely involve reading operations. Furthermore, the dataset is accessed only once during the multimodal training and prediction processes. As a result, we have removed the position remapping and block re-encryption features from PathORAM to simplify the access process. Thus, it increases the speed of dataset retrieval. As shown in Fig. 4, the MM_PathORAM algorithm is divided into two phases: the data initialization and reading phase.

$$k = \lfloor \log_2(n) \rfloor + 1 \quad (1)$$

During the data initialization phase (Lines 1 to 7 in Alg. 1), it initializes the position map $PM[n]$ based on n (dataset size), and the depth of the $PM[n]$ is k , as shown in Eq. (1). It maps the image-text pair data d_i into the real block R_i of a node (or bucket) B_i . the MM_PathORAM algorithm sets the capacity of B_i to 4 and randomly selects a region to

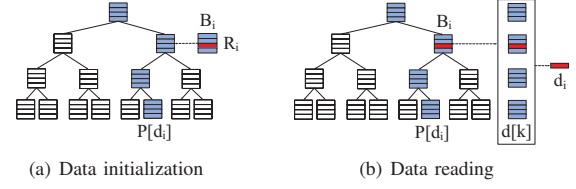


Fig. 4. MM_PathORAM algorithm.

Algorithm 1: MM_PathORAM algorithm

Input: All datasets $d[n]$
Output: Image-text pair data d_i

```

1 Initialize  $PM[n]$ ;
2 for  $d_i \in d[n]$  do
3    $R_i \leftarrow d_i$ ;
4    $B_i \leftarrow \text{random}(R_i) + 3 * U_i$ ;
5    $PM[n] \leftarrow \text{random\_node}(B_i)$ ;
6    $n = n - 1$ ;
7 Push  $PM[n]$  into enclave;
8 for  $d_i \in d[n]$  do
9    $P[d_i] = \text{oram.access}(PM[n], d_i)$ ;
10  if  $P[d_i]$  is not None then
11     $d[k] = \text{read\_data}(P[d_i])$ ;
12     $i = B_i - U_i$ ;
13     $d_i \leftarrow d[k]$ ;
14    return  $d_i$ .
15  else
16    print("Warning: file  $d_i$  does not exist");
17    return None.
```

store R_i using $\text{random}(R_i)$ function, as depicted in Fig. 4. Then, it fills the remaining three regions of B_i with dummy blocks U_i . A random node position is chosen for B_i using the random_node function, mapping it to the $PM[n]$. To prevent data overwriting issues when the MM_PathORAM algorithm writes to B_i again, this node will be marked as already written in the $PM[n]$. This process is repeated until all datasets are mapped into the $PM[n]$. Finally, the $PM[n]$ is stored in the SGX enclave.

We now analyze the time complexity of the data initialization phase. Initializing the $PM[n]$ is a linear operation with a time complexity of $O(n)$. The cost of randomly selecting a block in B_i to store data is $O(1)$. For the position map generation phase, it requires $n(n+1)/2$ iterations to map all $d[n]$ into the $PM[n]$, which costs $O(n^2)$. Thus, the overall time complexity of the data initialization phase is $O(n^2)$.

During the data reading phase (Lines 8 to 17 in Alg. 1), the process begins with the $\text{Oram.access}(PM[n], d_i)$ function retrieving the path $P[d_i]$ of all data blocks from the current leaf node to the root node in $PM[n]$. Then, based on the path $P[d_i]$, the corresponding k data blocks $d[k]$ are read from the disk, and the position i of the real data block R_i is obtained from B_i . Finally, the real data d_i is retrieved from $d[k]$ based

on the position i of the real data block.

The time complexity of the data reading phase consists of two parts. For the $Oram.access(PM[n], d_i)$ function, it costs $O(k)$, where k is the depth. It needs $O(1)$ to identify the real data block within node B_i . Thus, the total time consumption is $O(k)$.

Remarks. To protect data privacy, the mapped $PM[n]$ is always stored in SGX, and all operations on $PM[n]$ by the MM_PathORAM algorithm are carried out inside the enclave.

V. CONSTRUCTION OF SECMDP

The core of the SecMdp framework is the multimodal training and prediction process on the edge node. We utilize Occlum LibOS to build the multimodal training and prediction framework to tackle many aforementioned attack issues. In addition, the proposed MM_PathORAM algorithm is combined to establish a secure and efficient multimodal data loading channel from untrusted to trusted areas within SGX. It can defend against side-channel attacks during data loading in multimodal training and prediction processes. In this paper, we have employed the CLIP algorithm [38] within this framework.

A. Multimodal Training

As shown in Fig. 5, the multimodal training process in SecMdp is divided into three key phases: the transfer of data from the end node to the edge node, data training within the edge node and the transmission of model from the edge node to the cloud server. During the data and model transmission phases, a secure channel is established using the SGX remote attestation technology to defend against MITM attacks. The Occlum is utilized to protect against training data inference attacks, data poisoning attacks, model stealing attacks, model inversion attacks. Additionally, the MM_PathORAM algorithm is employed to load data from the disk, providing a defense against side-channel attacks. The flowchart of the algorithm is detailed in Alg. 2. Initially, an enclave is created within the edge node using Occlum, dividing the edge into a trusted area (enclave) and an untrusted external area (disk). Then, the SGX remote attestation technology provides secure data transmission between end and edge nodes. It relies on the Certificate Authority (CA) to generate a public key from the private key within the enclave and provide it to the user. The user encrypts data with this public key and uploads it to the edge node's disk. During data training phase, the enclave loads data from the untrusted area. To prevent side-channel attacks during data transmission, the MM_PathORAM algorithm sequentially loads training data into the enclave. Finally, the SGX remote attestation technology establishes a secure channel between the edge node and cloud server. The model trained at the edge node is encrypted transmitted to the cloud server through this channel for aggregation.

B. Multimodal Prediction

Unlike the multimodal training process, the multimodal prediction process primarily occurs between the edge and

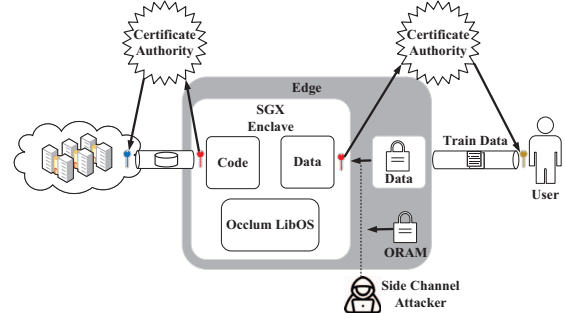


Fig. 5. Multimodal training in SecMdp framework.

Algorithm 2: Multimodal training algorithm in SecMdp framework

Input: $Train_Data$

Output: $Multimodel$

- 1 Initialize Occlum LibOS;
- 2 Set enclave $\leftarrow$ 6GB;
- 3 enclave $\leftarrow$ Code, Data;
- 4 Build Occlum LibOS;
- 5 Create CSR & Create private key;
- 6 User $Train_Data \rightarrow$ edge disk;
- 7 MM_PathORAM Read($Train_Data$);
- 8 CLIP Train($Train_Data$);
- 9 return $Multimodel \rightarrow$ cloud server.

end nodes. In this process, there is no information interaction with the cloud server, so we need not consider attacks originating from the cloud server. As illustrated in Fig. 6, the multimodal prediction process in SecMdp is divided into three key phases: data transmission from end node to edge node, data prediction within the edge node, and the transmission of prediction results from edge node to end node. During the data and result transmission phases, a secure channel is established using SGX remote attestation technology to defend against MITM attacks. The Occlum is utilized to protect against model stealing attacks and model inversion attacks. The MM_PathORAM algorithm is employed to de-

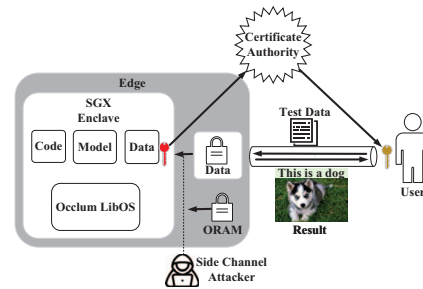


Fig. 6. Multimodal prediction in SecMdp framework.

Algorithm 3: Multimodal prediction algorithm in SecMdp framework

Input: *Test_Data***Output:** *PreResult*

```
1 Initialize Occlum LibOS;
2   Set enclave  $\leftarrow$  6GB;
3   enclave  $\leftarrow$  Code, Model, Data;
4 Build Occlum LibOS;
5 Greate CSR & Greate private key;
6 User upload Test_Data  $\rightarrow$  edge disk;
7 if Test_Data is not Null then
8   MM_PathORAM Read(Test_Data);
9   CLIP Prediction(Test_Data);
10  return PreResult  $\rightarrow$  end node.
```

fend against side-channel attacks when loading data from the disk. Differing from the multimodal training process, Occlum internally includes three components: *TestData*, *Code*, and *Model*. The *TestData* is used for prediction, the *Code* is the prediction code, and the *Model* refers to the model aggregated by the cloud server. The sizes of these components are 600M for *TestData*, 200K for *Code*, and 300M for *Model*, so the Occlum initializing an enclave space larger than 901M. The multimodal prediction algorithm flowchart in SecMdp is as shown in Alg. 3. Initially, the Occlum is initialized, setting the enclave size to 6G, allocating secure space for user data prediction. Simultaneously, the *Code*, *Data*, and *Model* files required for prediction transfer to the image directory of the new instance. The SGX remote attestation technology establishes a secure channel between edge and end nodes. Then users transmit the encrypted prediction data to the edge's disk. The enclave utilizes the MM_PathORAM algorithm to load data from the disk. Then, the data is decrypted within the enclave and used for plaintext state prediction inference. Finally, the prediction results are encrypted and transmitted to the end node.

VI. SECURITY ANALYSIS

In this section, we will analyze the security of the SecMdp framework. First, we analyze the security of the MM_PathORAM algorithm alongside the data loading process of the multimodal training and prediction algorithms. Then, we perform a security analysis of the multimodal training in SecMdp framework under the EEC architecture. Finally, we analyze the security of the multimodal prediction in SecMdp framework.

A. Security Analysis of MM_PathORAM

To prove the security of the MM_PathORAM algorithm, we conduct a security analysis by integrating it with the loading process of multimodal training and prediction algorithms. Initially, the dataset is accessed only once during the multimodal training and prediction processes, thereby reducing the probability of a successful attack. Furthermore, the $PM[n]$ of

the MM_PathORAM algorithm is stored in enclave, preventing attackers from accessing the $PM[n]$ information and further protecting data privacy.

Given a total of n image-text data pairs, the $PM[n]$ consists of n buckets, and the tree depth is $k = \lfloor \log_2(n) \rfloor + 1$. During the data initialization phase of the MM_PathORAM algorithm, the data paths are randomly stored in $PM[n]$ using two functions: $random(R_i)$ and $random_node(B_i)$. This operation is computationally indistinguishable. Thus, it secures the data mapping to $PM[n]$. During the data reading phase of the MM_PathORAM algorithm, the SGX retrieves $P[di]$ using the $Oram.access(PM[n], data)$ function. This phase is conducted internally within SGX. Thus, it prevents the untrusted edge node from accessing specific query steps. An untrusted edge node is susceptible to side-channel attacks. Attackers can only monitor SGX as it loads k encrypted blocks into the enclave. These encrypted blocks represented by the sequence $d[k]$, as shown in Eq. (2).

$$d[k] = \{d_k, d_{(k-1)}, \dots, d_1\} \quad (2)$$

In the sequence $d[k]$, only $d[i](0 < i < k)$ is the target data block. Since the data access pattern is oblivious, the edge node cannot determine the specific location of the target data during the retrieval process. During the process of reading $d[k]$ in the enclave, consider any two nodes $d[m]$, $d[n](0 < m, n < k \ \& \ m \neq n)$ chosen from $d[k]$. The probability of the attacker accessing $d[m]$ is represented as $Pr(d[m])$, and $Pr(d[m]) = Pr(d[n]) = \frac{1}{k}$. The position of the data blocks stored in each node is independent. Even if an attacker deduces the data information of $d[m]$, it will not provide any additional information about $d[n]$. There are four data blocks in $d[i]$ and only one is a real block. Thus, the probability of an edge node correctly inferring the real block is $\frac{1}{4k}$. Finally, as data is loaded into the enclave and dummy blocks are removed, the edge node cannot monitor the data operation workflow.

B. Security Analysis of the Multimodal Training in SecMdp

To verify the security of the multimodal training in SecMdp framework, we analyze the security of the framework from the EEC architecture. To preserve the security of data and model, the SGX on the edge node isolates the cloud server and the end node. Thus, the cloud server cannot directly access the data at the end node, and the end node cannot access the aggregated model on the cloud server. We establish a secure channel between the end node and cloud server using the SGX remote attestation mechanism in the edge node. It ensures that the end node and cloud server exchange data only with a verified and trusted edge node. Additionally, data and models within the EEC architecture are encrypted and stored with enclave-specific keys. Thus, it can effectively prevent MITM attacks during transmission.

The multimodal training process is conducted in the enclave, which protects the process of code execution. Attackers cannot infer data content or tamper with the program through data inference attacks, model stealing attacks, and model inversion

attacks. Thus, it guarantees the integrity and confidentiality of the computation process. During the process of reading disk data by the multimodal training algorithm, the training datasets are encrypted and stored. It achieves data integrity and provides robust defense against data poisoning attacks. For side-channel attacks that SGX cannot prevent, the MM_PathORAM algorithm makes the memory access pattern oblivious. This prevents attackers from inferring data or model information by observing cache behavior, effectively defending against cache-based side-channel attacks. It also prevents the analysis of data characteristics through observing page faults, reducing the risk of page fault attacks.

Thus, the multimodal training in SecMdp framework can effectively defend against various types of attacks in the EEC architecture, and it can secure both data and model.

C. Security Analysis of the Multimodal Prediction in SecMdp

The multimodal prediction in SecMdp framework can enhance the security and privacy of data and model. During the multimodal training phase, the model aggregated in the cloud server has been deployed to edge nodes. The multimodal prediction process occurs only between the end nodes and edge nodes. The cloud server does not need to access data or provide computational power, which eliminates concerns about MITM attacks originating from the cloud server. Furthermore, to preserve data prediction security, the framework deploys the data, the model and the code within the enclave. Consequently, the data prediction process becomes invisible to the outside, thereby enabling the framework to defend against model stealing and model inversion attacks. Additionally, using the SGX remote attestation mechanism builds trust from the end node to the edge node. Encrypting multimodal prediction data and results also prevents information leakage during transmission. Finally, employing the MM_PathORAM algorithm to load prediction data from the disk effectively prevents cache-based side-channel attacks and page fault attacks.

To conclude, the multimodal prediction in SecMdp framework can effectively protect the security of data and models, and it can defend against various types of attacks in the multimodal prediction process.

VII. EVALUATION

To assess the general applicability of SecMdp, we select three widely recognized datasets in computer vision: COCO [41], PASCAL Visual Object Classes(VOC) [40] and CIFAR-100 [42], for conducting multimodal training and prediction tasks. Since the COCO dataset has extensive multimodal content, we have selected it as our training dataset. The performance of the multimodal prediction in SecMdp framework is evaluated using the COCO, VOC and CIFAR-100 datasets concurrently.

The experimental environment of our implementation is Ubuntu20.04 system, CPU is Intel(R) Xeon(R) W-1290P@3.70GHz, 64GB memory, and the graphics card is NVIDIA 3090ti.

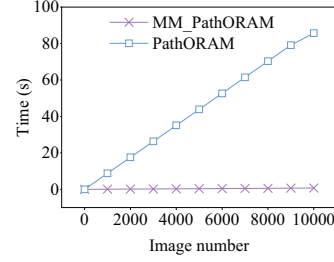


Fig. 7. MM_PathORAM data reading process.

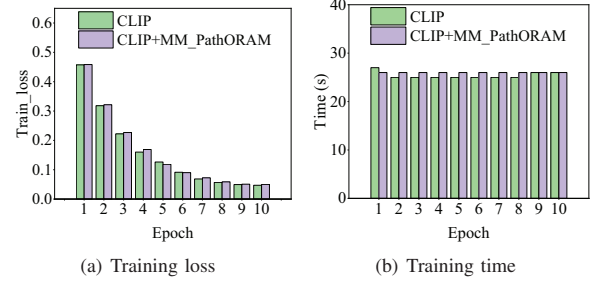


Fig. 8. Comparison of training loss and training time between the multimodal training based on MM_PathORAM and the CLIP algorithm.

A. MM_PathORAM

We evaluate the data loading speed of the MM_PathORAM algorithm through comparative experiments with the PathORAM algorithm on the COCO dataset. The effectiveness of MM_PathORAM algorithm in multimodal training is assessed by its performance in data loading, as indicated by loss values and training time. Furthermore, we assess the generalizability of the MM_PathORAM algorithm within the multimodal prediction process, which analyzes prediction accuracy and processing time across various datasets. Finally, we have made the MM_PathORAM source code publicly accessible to allow for further assessment of our experimental results.¹

1) Data Loading Speed of MM_PathORAM Algorithm:

Fig. 7 shows the performance of the PathORAM algorithm, which degrades with increasing data volume due to the need to access more tree nodes for data reading. For instance, loading 10,000 images with PathORAM takes up to 85.71 seconds. However, the MM_PathORAM algorithm demonstrates a remarkable efficiency, requiring only 0.73 seconds to load the same number of images, thereby achieving a speed enhancement of 117 times.

2) Effectiveness of MM_PathORAM Algorithm:

Fig. 8(a) presents a comparison of the training loss values between the CLIP algorithm and the MM_PathORAM algorithm used for data loading. The observed difference in loss values remains below 0.02 throughout the training, demonstrating that MM_PathORAM algorithm does not affect the loss in multimodal training processes. Fig. 8(b) presents a compar-

¹<https://github.com/ZhaoBaigit/MMPPathORAM>

TABLE III
COMPARISON OF PREDICTION ACCURACY ACROSS DIFFERENT DATA
LOADING ALGORITHMS

	COCO	VOC	Cifar-100
CLIP Accuracy	94.4%	95.7%	86.8%
CLIP+MM_PathORAM Accuracy	94.4%	95.7%	86.8%

ison of the CLIP algorithm with and without employing MM_PathORAM algorithm for data loading. The difference in data loading time per epoch between the two scenarios is within one second. The experimental results indicate that the application of MM_PathORAM algorithm in data loading has a negligible effect on both the loss and the speed of multimodal training. Thereby validating its efficacy in the context of data training.

3) *Generalizability of MM_PathORAM Algorithm*: Fig. 9 presents a comparative analysis of the prediction times for the CLIP algorithm, comparing its performance when utilizing the MM_PathORAM algorithm for data loading against its performance without it. Fig. 9(a), Fig. 9(b) and Fig. 9(c) depict the prediction time line graphs on the COCO, VOC and Cifar100 datasets, respectively. Ten categories from each dataset were randomly chosen, with each containing 100 to 800 images for prediction. The observed time difference in algorithm performance across these datasets remains under four seconds. These results suggest that using MM_PathORAM algorithm in the multimodal prediction algorithm does not reduce the data loading rate during prediction. Table III shows results from multimodal prediction experiments conducted on different datasets, comparing the two algorithms. The MM_PathORAM-based framework achieves comparable accuracy to the CLIP algorithm across multiple datasets. This shows that loading data through the MM_PathORAM algorithm does not affect prediction accuracy. This evidence corroborates the strong generalizability of the MM_PathORAM algorithm.

B. Multimodal Data Training in SecMdp

1) *Multimodal Data Training Evaluation Metrics in SecMdp*: To verify the effectiveness of the multimodal training in SecMdp framework, we assess its impact on the training process through loss values. Training loss values are crucial for evaluating the learning status and performance of model, and for understanding the influence of the operational environment on model training.

The memory utilization ratio highlights memory usage during the multimodal training process. Achieving an optimal balance in memory utilization is crucial; neither excessive nor insufficient utilization is beneficial. High memory utilization indicates significant consumption of memory resources by the program, which is an important aspect to monitor. Excessive memory consumption not only has the potential to deprive other applications of essential memory resources but also to trigger system page swapping, which could adversely impact

the overall performance of the system. Moreover, high memory usage can become a target for attackers, who might exploit it to conduct page fault attacks, posing a risk to the integrity of user data. Conversely, minimal memory utilization could suggest that the available memory is not being fully utilized, potentially leading to poor performance.

The memory utilization calculation is shown in Eq. (3), where the MU represents the memory utilization, the $MemUsed$ represents the used amount of physical memory, and the $MemTotal$ represents the total physical memory.

$$MU = \frac{MemUsed}{MemTotal} \quad (3)$$

The memory utilization fluctuates during program execution. Therefore, we introduce the average memory utilization as a primary evaluation metric, as shown in Eq. (4). The AV_MU represents the average memory utilization, the MU_{max} represents the maximum memory utilization, and the MU_{min} represents the minimum memory utilization.

$$AV_MU = \frac{MU_{max} + MU_{min}}{2} \quad (4)$$

2) *Multimodal Data Training Experimental Results in SecMdp*: The graph in Fig. 10 presents the loss values observed during the training process under three distinct scenarios: CLIP, CLIP+Occlum and CLIP+SecMdp. The graph reveals minor fluctuations in loss values, which can be attributed to factors such as random initialization, the random shuffling of data, and the influence of regularization techniques. These fluctuations are typical in deep learning models and do not signify any issues with the training process. The consistent loss values across different scenarios demonstrate that the implementation of this framework does not adversely impact the effectiveness of the multimodal training process.

The memory space within the SGX enclave is isolated from processes and the operating system, $MemTotal$ is 6GB. As shown in Table IV, the memory utilization rate during the multimodal training process in the CLIP algorithm under three algorithmic frameworks exhibits AV_MU of 37.0%, 39.0% and 37.5%, respectively. The average memory utilization under the SecMdp framework is 0.5% higher than the CLIP algorithm without any framework and 1.5% lower than the Occlum framework, with a variation within 2%. This indicates that the impact of the framework on the memory utilization rate during multimodal training is minimal. Memory utilization for the CLIP algorithm fluctuates by 16%, while for CLIP+Occlum, the fluctuation is reduced to 2%, and for CLIP+SecMdp, it is 5%. The design strategy of Occlum enables more efficient and stable management and usage of memory, reducing fluctuations in memory utilization. The wider fluctuation range in memory utilization under the SecMdp framework, in comparison to CLIP+Occlum, mainly stems from the MM_pathORAM algorithm, which necessitates loading 16 buckets simultaneously. This requirement leads to greater variation in memory consumption. Nonetheless, the SecMdp framework effectively counters side-channel attacks,

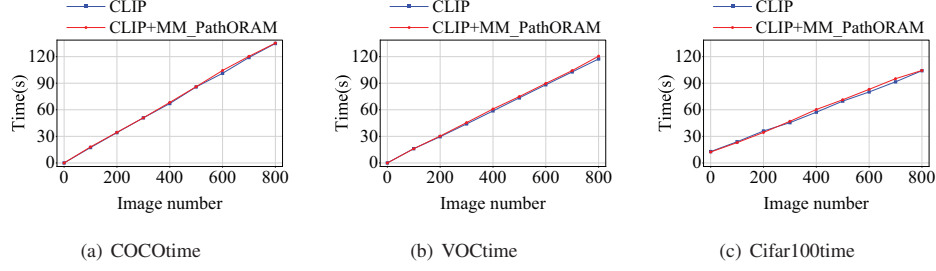


Fig. 9. Comparison of prediction time between the multimodal prediction framework based on the MM_PathORAM and CLIP algorithm.

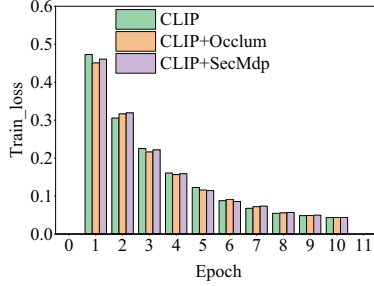


Fig. 10. Training loss of CLIP, CLIP+Occlum, CLIP+SecMdp.

TABLE IV
COMPARISON OF MEMORY UTILIZATION RATES IN DIFFERENT DATA TRAINING FRAMEWORKS

Algorithm framework	MemUsed (GB)	MemTotal (GB)	MU	AV_MU
CLIP	18.3 – 28.8	64.0	29% – 45%	37.0%
CLIP+Occlum	2.3 – 2.4	6.0	38% – 40%	39.0%
CLIP+SecMdp	2.1 – 2.4	6.0	35% – 40%	37.5%

ensuring data and code security. Therefore, considering memory efficiency and security, employing the SecMdp framework for multimodal training programs is the optimal choice.

The results show that the training loss of the multimodal data training in SecMdp framework only differs by 0.8%, affirming the effectiveness of framework. The AV_MU of the SecMdp framework is 0.5% higher than the CLIP algorithm, but its memory fluctuation is 11% lower than CLIP algorithm, indicating a more efficient and stable management and use of memory. Therefore, the SecMdp framework produces the best results for the multimodal training program, considering training loss, memory efficiency and security.

C. Multimodal Prediction in SecMdp

1) Multimodal Prediction Evaluation Metrics in SecMdp:

In the multimodal prediction process within the SecMdp framework, we evaluate the performance using two key indicators: runtime and the accuracy of the multimodal CLIP algorithm. The accuracy evaluates whether this framework affects the performance of multimodal predictions. The runtime assesses whether the framework can meet the demands of real-world applications.

The accuracy calculation formula is shown in Eq. (5). Here, the $Accuracy$ represents the CLIP algorithm prediction accuracy rate, the C_{num} represents the number of correctly predicted samples, and the S_{num} represents the total number of samples.

$$Accuracy = \frac{C_{num}}{S_{num}} \quad (5)$$

The average accuracy calculation formula is as shown in Eq. (6). Here, the AVG_Acc represents the average accuracy, the $\sum_{i=1}^N A_i$ represents the sum of the accuracy rates of all samples, and the $Classnum$ represents the number of sample categories.

$$AVG_Acc = \frac{\sum_{i=1}^N A_i}{Classnum} \quad (6)$$

2) Multimodal Data Prediction Experimental Results in SecMdp: The prediction process of the CLIP algorithm is as follows. A random selection of 10 data categories from the CIFAR-100 public dataset undergoes testing. These categories include “apple”, “fish”, “baby”, “bear”, “beaver”, “bed”, “bee”, “beetle”, “bicycle” and “bottle”. Each category contains 100 images, with pixel dimensions of 32x32. We set a confidence level of 50%, meaning samples with accuracy rates greater than 50% are considered as positive samples. The labels are then semantically translated, for instance, the label “apple” is rephrased as “This is an apple”. The CLIP model performs image-text matching, outputting the highest accuracy rate as the final prediction result.

To assess the efficiency of the multimodal prediction phase, we evaluate the accuracy of three variations of the CLIP algorithm: the standard CLIP algorithm, the SGX-assisted CLIP algorithm, and the SecMdp-based CLIP algorithm. The accuracy rates of the ten sample categories from the experiment are shown in Table V. Specifically, we tested the accuracy of CLIP algorithm, CLIP algorithm with SGX protection, and CLIP+SecMdp. The average accuracy of the multimodal CLIP algorithm is 85.3%, The prediction accuracy of CLIP algorithm within SGX is 85.2%, and the accuracy of the CLIP algorithm, based on the SecMdp framework, is 85.4%. The average accuracy has only a 0.2% difference, which is within the expected range. The results indicate that the SecMdp

TABLE V
ACCURACY OF DIFFERENT CATEGORIES

Data Categories	CLIP Accuracy	CLIP+Occlum Accuracy	CLIP+SecMdp Accuracy
apple	99%	99%	99%
fish	98%	98%	98%
baby	84%	84%	85%
bear	65%	65%	65%
beaver	72%	71%	72%
bed	66%	66%	66%
bee	87%	87%	87%
beetle	97%	97%	97%
bicycle	89%	89%	89%
bottle	96%	96%	96%
AVG_Acc	85.3%	85.2%	85.4%

TABLE VI
ACCURACY OF DIFFERENT DATASETS

Different Datasets	CLIP Accuracy	CLIP+Occlum Accuracy	CLIP+SecMdp Accuracy
Cifar-100	92.9%	92.9%	93.0%
VOC2012	90.4%	90.5%	90.5%
COCO	89.4%	89.4%	89.3%

design does not affect the accuracy of the prediction phase, comparing with the systems without privacy protection.

To evaluate the versatility of the framework, we conducted an experiment using 1000 images randomly selected from the VOC2012 and COCO datasets. These images were introduced into the multimodal prediction framework built on Occlum LibOS for validation prediction. The outcomes of this evaluation are presented in Table VI. A discernible trend emerges from the data: the mean accuracy across diverse datasets exhibits coherence. This indicates that predictions made on varied datasets retain a robust level of generalization within this framework. In comparing the intrinsic characteristics of the datasets, the VOC and COCO datasets have a pixel count roughly 10-20 times greater than CIFAR-100. The VOC and COCO datasets have pixel counts approximately 10-20 times greater than that of CIFAR-100, and their images contain multiple objects, thus resulting in slightly lower prediction accuracy compared to CIFAR-100.

The execution duration for various data types is presented in Table VII. Considering that predictions in real-world applications are typically executed in smaller batches, we established the average prediction time per image as the primary metric for evaluation. We calculated the average prediction time using a sample of 1000 images. Since the images in the Cifar-100 dataset have a lower resolution and a simpler background, the average prediction time for this dataset is 0.1s less than for the VOC and COCO datasets. Taking the Cifar-100 dataset as an example, in both the Occlum LibOS and SecMdp frameworks based CLIP prediction algorithm, the average prediction time per image is 4.1 seconds. In a non-Occlum LibOS envi-

TABLE VII
TIME CONSUMPTION FOR DIFFERENT DATASETS

Different Datasets	CLIP Time	CLIP+Occlum Time	CLIP+SecMdp Time
Cifar-100	0.2s	4.1s	4.1s
VOC2012	0.3s	4.2s	4.2s
COCO	0.3s	4.2s	4.2s

ronment, the average time for predicting each image is 0.2 seconds. The primary reason is that the enclave size in Occlum LibOS and SecMdp frameworks is 6G, whereas the memory available in a non-Occlum LibOS environment is 64G, leading to a ten-fold difference in computing power. Also, the internal structure of SecMdp frameworks and Occlum LibOS consume memory. Thus, the prediction speed based on the SecMdp framework is 3.9 seconds slower than in regular circumstances. This framework guarantees the security of predictive data at the expense of 3.9 seconds, and the prediction time of 4.1 seconds per image can meet the practical production requirements.

The results show that the accuracy of CLIP within SecMdp framework is only 0.1% different from the CLIP algorithm. Furthermore, it only uses 6GB of memory to complete a single image prediction in 4.1 seconds. In terms of accuracy, speed and security, the multimodal prediction algorithm in SecMdp performs best.

VIII. CONCLUSION

In this paper, we developed a multimodal deep learning framework, SecMdp, utilizing the EEC architecture. By adopting the SGX, SecMdp can protect data privacy and defend against prevalent attacks in the EEC architecture. To deal with the side-channel attacks for SGX, we designed a tailored PathORAM algorithm for multimodal applications. Our design robustly protects multimodal data throughout the loading, training, and prediction stages within the EEC architecture. Experimental results confirm that this framework maintains the efficiency of multimodal data loading, the integrity of multimodal training parameters, and the accuracy of multimodal prediction. The proposed SecMdp framework has excellent portability and can be used in other deep learning algorithms and EEC architecture requiring security scenarios.

ACKNOWLEDGMENT

This work was supported by the National Science and Technology Major Project (No. 2022ZD0115901), in part by the National Natural Science Foundation of China (No. 62177007, No. 62102035, No. 71961022, No. 62302485, No. 62202398), the Research Grants Council of Hong Kong under Grants CityU (No. 11213920, No. R1012-21), the Guangdong Basic and Applied Basic Research Foundation (No. 2023A1515140137), and the China Postdoctoral Science Foundation (No. 2022M713206).

REFERENCES

- [1] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," unpublished, arXiv preprint arXiv:2303.12712, 2023.
- [2] X. Yu, H. Zhang, R. Hong, Y. Song, and C. Zhang, "VD-PCR: Improving visual dialog with pronoun coreference resolution," *Pattern Recognition*, vol. 125, pp. 108540, 2022.
- [3] Y. Almalioglu, M. Turan, N. Trigoni, and A. Markham, "Deep learning-based robust positioning for all-weather autonomous driving," *Nature Machine Intelligence*, vol. 4, no. 9, pp. 749-760, 2022.
- [4] J. Xie, Z. Zhao, Z. Lin, and Y. Shen, "Multimodal Graph Learning for Cross-Modal Retrieval," in *Proceedings of the SIAM International Conference on Data Mining (SDM)*, Society for Industrial and Applied Mathematics, pp. 145-153, 2023.
- [5] S. Yang, Z. Zhang, C. Zhao, X. Song, S. Guo, and H. Li, "CNNPC: End-Edge-Cloud collaborative CNN inference with joint model partition and compression," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 4039-4056, 2022.
- [6] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with Edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738-1762, 2019.
- [7] Y. Li, D. Zeng, L. Gu, Q. Chen, S. Guo, A. Zomaya, et al., "Lasagna: Accelerating secure deep learning inference in sgx-enabled Edge Cloud," in *Proc. ACM Symposium on Cloud Computing*, pp. 533-545, 2021.
- [8] M. Conti, N. Dragoni, and V. Lesyk, "A survey of man in the middle attacks," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 3, pp. 2027-2051, 2016.
- [9] Z. Yang, X. He, Z. Li, M. Backes, M. Humbert, P. Berrang, et al., "Data poisoning attacks against multimodal encoders," in *International Conference on Machine Learning*, PMLR, pp. 39299-39313, July 2023.
- [10] X. Gong, Y. Chen, Q. Wang, M. Wang, and S. Li, "Private data inference attacks against Cloud: Model, technologies, and research directions," *IEEE Communications Magazine*, vol. 60, no. 9, pp. 46-52, 2022.
- [11] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 1322-1333, 2015.
- [12] T. Lee, Z. Lin, S. Pushp, C. Li, Y. Liu, Y. Lee, et al., "Occlumency: Privacy-preserving remote deep-learning inference using SGX," in *The 25th Annual International Conference on Mobile Computing and Networking*, pp. 1-17, October 2019.
- [13] A. Ito, K. Saito, R. Ueno, and N. Homma, "Imbalanced data problems in deep learning-based side-channel attacks: Analysis and solution," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3790-3802, 2021.
- [14] A. Ahmad, K. Kim, M. I. Sarfaraz, and B. Lee, "OBLIVIAE: A Data Oblivious Filesystem for Intel SGX," in *NDSS*, February 2018.
- [15] E. Stefanov, M. V. Dijk, E. Shi, T. H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: an extremely simple oblivious RAM protocol," in *Proc. of CCS*, pp. 299-310, 2013.
- [16] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, et al., "VQA: Visual question answering," in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2425-2433, 2015.
- [17] A. Das, S. Kottur, K. Gupta, A. Singh, D. Yadav, J. M. Moura, et al., "Visual dialog," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 326-335, 2017.
- [18] R. Zellers, Y. Bisk, A. Farhadi, and Y. Choi, "From recognition to cognition: Visual commonsense reasoning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6720-6731, 2019.
- [19] A. Karpathy and L. Fei-Fei, "Deep visual-semantic alignments for generating image descriptions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3128-3137, 2015.
- [20] D. Kim, C. Guyot, "Optimized Privacy-Preserving CNN Inference With Fully Homomorphic Encryption," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2175-2187, 2023.
- [21] Q. Lou, W. J. Lu, C. Hong, and L. Jiang, "Falcon: Fast Spectral Inference on Encrypted Data," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 2364-2374, 2020.
- [22] J. Feng, L. T. Yang, B. Ren, D. Zou, M. Dong, and S. Zhang, "Tensor Recurrent Neural Network with Differential Privacy," in *IEEE Transactions on Computers*, 2023.
- [23] L. Yin, S. Lin, Z. Sun, S. Wang, R. Li, and Y. He, "PriMonitor: An adaptive tuning privacy-preserving approach for multimodal emotion detection," *World Wide Web*, vol. 27, no. 2, pp. 1-28, 2024.
- [24] W. Zheng, Y. Wu, X. Wu, C. Feng, Y. Sui, X. Luo, and Y. Zhou, "A survey of Intel SGX and its applications," *Frontiers of Computer Science*, vol. 15, pp. 1-15, 2021.
- [25] K. Ren, Y. Guo, J. Li, X. Jia, C. Wang, Y. Zhou, et al., "HybridIX: New hybrid index for volume-hiding range queries in data outsourcing services," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, pp. 23-33, 2020.
- [26] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, et al., "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 955-970, 2020.
- [27] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: protecting against DNN model stealing attacks," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 512-527, 2019.
- [28] O. Goldreich and R. Ostrovsky, "Software protection and simulation on oblivious RAMs," *Journal of the ACM (JACM)*, vol. 43, no. 3, pp. 431-473, 1996.
- [29] W. Chen, T. Hoang, J. Guajardo, and A. A. Yavuz, "Titanium: A metadata-hiding file-sharing system with malicious security," in *Network and Distributed System Security (NDSS) Symposium*, 2022.
- [30] R. Rajat, Y. Wang, and M. Annavaram, "Laoram: A look ahead oram architecture for training large embedding tables," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pp. 1-15, 2023.
- [31] M. Raoufi, Y. Zhang, and J. Yang, "Ir-oram: Path access type based memory intensity reduction for path-oram," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 360-372, 2022.
- [32] R. Wang, Y. Zhang, and J. Yang, "D-oram: Path-oram delegation for low execution interference on Cloud servers with untrusted memory," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 416-427, 2018.
- [33] F. Brasser, U. Müller, A. Dmitrienko, K. Kostianen, S. Capkun, and A. R. Sadeghi, "Software grand exposure: SGX cache attacks are practical," in *11th USENIX Workshop on Offensive Technologies (WOOT 17)*, 2017.
- [34] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *IEEE Symposium on Security and Privacy*, pp. 640-656, May 2015.
- [35] J. Niu, W. Peng, X. Zhang, Y. Zhang, et al., "Narrator: Secure and practical state continuity for trusted execution in the cloud," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, pp. 2385-2399, 2022.
- [36] S. Matetic, M. Ahmed, K. Kostianen, A. Dhar, D. Sommer, A. Gervais, et al., "ROTE: Rollback protection for trusted execution," in *26th USENIX Security Symposium (USENIX Security 17)*, pp. 1289-1306, 2017.
- [37] R. Strackx and F. Piessens, "Ariadne: A minimal approach to state continuity," in *25th USENIX Security Symposium (USENIX Security 16)*, pp. 875-892, 2016.
- [38] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., "Learning transferable visual models from natural language supervision," in *International Conference on Machine Learning, PMLR*, pp. 8748-8763, 2021.
- [39] M. Sabt, M. Achemlal, and A. Bouabdallah, "Trusted execution environment: what it is, and what it is not," in *Proceedings of IEEE Trustcom/BigDataSE/Ispas*, vol. 1, pp. 57-64, August 2015.
- [40] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, pp. 98-136, 2015.
- [41] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision-ECCV: 13th European Conference, Zurich, Switzerland, Proceedings, Part V*, pp. 740-755, Springer International Publishing, September 2014.
- [42] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," 2009.